

AETHER — Urban Air Quality Intelligence Platform

Solo Build Plan | ET AI Hackathon 2026 | Problem Statement 5

Version: 1.0

City: Kolkata, West Bengal, India

Build Mode: Solo / Vibe-Coded

Timeline: 10-12 Days

Primary Author: [Your Name]

Date: 2026-06-22

1. Executive Summary

Product: AETHER (Air-quality Evaluation, Tracking & Hyperlocal Environmental Response)

Tagline: *“From measurement to intervention — intelligence that cleans the air.”*

AETHER is a geospatial AI platform that fuses live CPCB station data, satellite NO₂ layers, weather forecasts, and OpenStreetMap land-use data into a single command center. It does not just display AQI — it explains WHY a ward is polluted, forecasts WHAT it will be in 24–72 hours, and tells enforcement teams exactly WHERE to act next.

Why Kolkata: The builder lives in Kolkata. The city has 10+ active CPCB stations, distinct pollution signatures (Howrah traffic corridor, winter inversion layer, brick kiln emissions in the northeast, New Town construction dust), and 144 well-defined municipal wards. Every other team will pick Delhi. Kolkata, with deep local context, will stand out.

2. Strategic Scope

The Spine (Build Deep, Real, Working)

- Live Geospatial AQI Map** — Real CPCB station data on a real Kolkata map, refreshed on schedule.
- Hyperlocal Forecasting Agent** — Real XGBoost model predicting AQI 24–72h ahead per zone, validated against held-out data.
- Source Attribution Layer** — Explainable scoring model attributing pollution to traffic / industrial / construction / biomass / residential with confidence.
- Enforcement Priority Ranking** — Ranked, justified action list for inspectors.

Satellite Modules (Build Lighter, But Real)

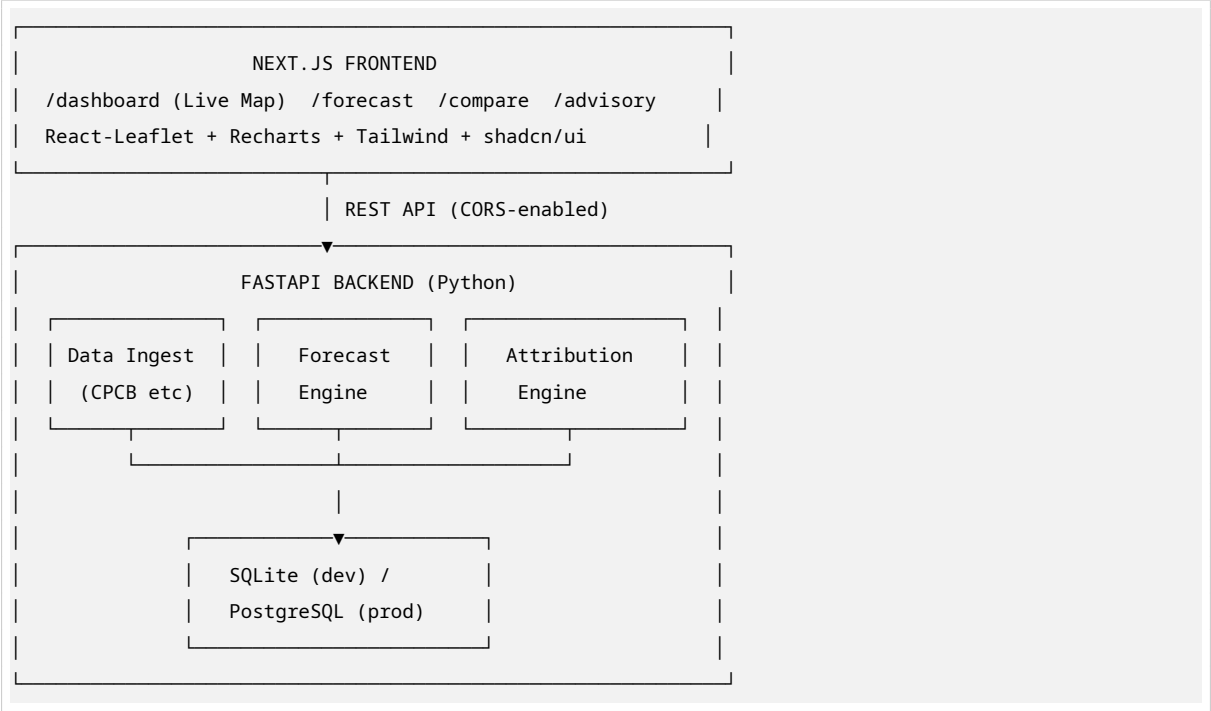
- Citizen Health Advisory** — Generated text alerts per ward in English + Bengali.
- Multi-City Comparison** — Same pipeline run for Delhi and Mumbai to prove generalizability.
- Satellite NO₂ Overlay** — Real Sentinel-5P data via Google Earth Engine as corroborating layer.

Explicitly Out of Scope (Do Not Build)

- IVR system
- WhatsApp Business API integration
- Real-time municipal system integration

- True 1km atmospheric dispersion modelling
- Multi-agent orchestration frameworks (CrewAI, AutoGen, etc.)

3. System Architecture



Design Principle: The LLM narrates computed numbers. It never generates the numbers themselves. All AQI forecasts, attribution scores, and priority rankings are computed by deterministic Python code. The LLM only writes natural-language explanations.

4. Tech Stack

Layer	Technology	Justification
Frontend Framework	Next.js 14 (App Router) + TypeScript	SSR, API routes, Vercel deploy, AI tools generate reliably
Styling	Tailwind CSS + shadcn/ui	Copy-paste beautiful components, zero design skill
Maps	React-Leaflet + CartoDB Dark Matter tiles	Free, no API key, looks premium instantly
Charts	Recharts	Declarative React charts, easy to vibe-code
Backend	FastAPI + Uvicorn	Async, auto-docs, Python ML ecosystem
Database (Dev)	SQLite	Zero setup, portable, single file
Database (Prod)	PostgreSQL + PostGIS (Supabase)	Managed, geospatial queries, free tier
ORM	SQLAlchemy 2.0	Familiar, works with SQLite and Postgres
Forecasting	XGBoost + scikit-learn	Faster to train than LSTM, easier to explain to judges, validated RMSE

Table 1 – continued

Layer	Technology	Justification
Attribution	Rule-based + statistical scoring	Explainable, defensible, no black-box ML needed
Satellite	Google Earth Engine (Sentinel-5P)	Real data, free for research, ~5.5km resolution
Weather	Open-Meteo API	Free, no key required, unlimited
RAG/LLM	LangChain + OpenAI GPT-4o-mini	Cheap, fast, multilingual (English + Bengali)
OSM Data	osmnx / Overpass API	Free road networks, industrial zones, POIs
Deploy	Vercel (frontend) + Render/Railway (backend)	Git push → live, free tiers generous

5. Database Schema

5.1 stations

Column	Type	Description
id	INTEGER PK	Auto-increment
station_code	TEXT UNIQUE	CPCB station identifier
name	TEXT	Station name (e.g., “Victoria Memorial”)
lat	REAL	Latitude
lon	REAL	Longitude
ward_id	INTEGER FK	Reference to wards table
city	TEXT DEFAULT ‘Kolkata’	City name

5.2 readings (Time-Series)

Column	Type	Description
id	INTEGER PK	Auto-increment
station_id	INTEGER FK	Reference to stations
measured_at	TIMESTAMP	Observation timestamp
pm25	REAL	PM2.5 $\mu\text{g}/\text{m}^3$
pm10	REAL	PM10 $\mu\text{g}/\text{m}^3$
no2	REAL	NO $\mu\text{g}/\text{m}^3$
so2	REAL	SO $\mu\text{g}/\text{m}^3$
co	REAL	CO mg/m^3
o3	REAL	O $\mu\text{g}/\text{m}^3$
aqi	REAL	Computed overall AQI
category	TEXT	‘Good’, ‘Satisfactory’, ‘Moderate’, ‘Poor’, ‘Very Poor’, ‘Severe’

5.3 weather

Column	Type	Description
id	INTEGER PK	Auto-increment
city	TEXT	City name
measured_at	TIMESTAMP	Timestamp
temp_c	REAL	Temperature °C
humidity_pct	REAL	Relative humidity %
wind_speed	REAL	Wind speed km/h
wind_dir	REAL	Wind direction degrees
pressure	REAL	Pressure hPa

5.4 wards (KMC Administrative Boundaries)

Column	Type	Description
id	INTEGER PK	Auto-increment
ward_no	INTEGER	KMC ward number (1-144)
ward_name	TEXT	Ward name
lat_center	REAL	Centroid latitude
lon_center	REAL	Centroid longitude
geojson	TEXT	Boundary polygon as JSON string
population	INTEGER	Census population
area_sqkm	REAL	Area in km ²
road_density	REAL	km road per km ²
industrial_score	REAL	0-100 proximity to industrial zones
construction_count	INTEGER	Active construction sites (OSM)
school_count	INTEGER	Schools in ward (OSM)
hospital_count	INTEGER	Hospitals in ward (OSM)

5.5 forecasts

Column	Type	Description
id	INTEGER PK	Auto-increment
station_id	INTEGER FK	Nearest station
ward_id	INTEGER FK	Target ward
forecast_for	TIMESTAMP	Prediction timestamp
predicted_aqi	REAL	Predicted AQI value
confidence_lower	REAL	Lower bound
confidence_upper	REAL	Upper bound
predicted_category	TEXT	Predicted category
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Generation time

5.6 attributions

Column	Type	Description
id	INTEGER PK	Auto-increment
ward_id	INTEGER FK	Target ward
computed_at	TIMESTAMP	Computation time
traffic_pct	REAL	Traffic contribution %
industrial_pct	REAL	Industrial contribution %
construction_pct	REAL	Construction contribution %
residential_pct	REAL	Residential contribution %
biomass_pct	REAL	Biomass burning contribution %
confidence	REAL	Overall confidence 0-1
primary_source	TEXT	Dominant source category
explanation	TEXT	Human-readable explanation

5.7 enforcement_queue

Column	Type	Description
id	INTEGER PK	Auto-increment
ward_id	INTEGER FK	Target ward
priority_score	REAL	Computed priority 0-100
action_text	TEXT	Recommended action
target_type	TEXT	‘construction’, ‘industrial’, ‘traffic’, ‘biomass’
status	TEXT DEFAULT ‘open’	‘open’, ‘deployed’, ‘resolved’
created_at	TIMESTAMP DEFAULT CUR- RENT_TIMESTAMP	Creation time
resolved_at	TIMESTAMP	Resolution time

5.8 documents (RAG Corpus)

Column	Type	Description
id	INTEGER PK	Auto-increment
title	TEXT	Document title
content	TEXT	Full text
source	TEXT	Source (CPCB, WHO, etc.)
doc_type	TEXT	‘health_guideline’, ‘cpcb_advisory’, ‘research’
embedding	BLOB	Vector embedding (1536-dim)
metadata	TEXT	JSON metadata

5.9 advisory_logs

Column	Type	Description
id	INTEGER PK	Auto-increment
session_id	TEXT	Chat session ID
question	TEXT	User question
answer	TEXT	Generated answer
language	TEXT	'en', 'bn', 'hi'
lat	REAL	User location lat
lon	REAL	User location lon
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Time

6. API Design (FastAPI Endpoints)

6.1 Core Data Endpoints

Method	Endpoint	Query Params	Response	Description
GET	/api/health	—	{"status": "ok", "version": "1.0.0", "city": "Kolkata"}	Service health check
GET	/api/cities	—	[{"id": "kolkata", "name": "Kolkata", "lat": 22.5726, "lon": 88.3639}]	List configured cities
GET	/api/stations	?city=Kolkata	Array of station objects	All stations for city
GET	/api/aqi/live	?city=Kolkata	Latest reading per station	Current AQI snapshot
GET	/api/aqi/heatmap	?city=Kolkata	Grid of ward centers + interpolated AQI	Map coloring data
GET	/api/wards	?city=Kolkata	Array of ward objects with boundaries	Ward geometry
GET	/api/wards/ {ward_id}	—	Ward detail + current AQI + attribution	Single ward deep-dive

6.2 Intelligence Endpoints

Method	Endpoint	Body / Query	Response	Description
GET	/api/forecast	?lat=22.57&lon=88.36&hours=72	Hourly predictions for 72h	AQI forecast for location
GET	/api/attribution/{ward_id}	—	Source breakdown + confidence + explanation	Why is this ward polluted?
GET	/api/enforcement	?city=Kolkata&limit=10	Ranked action list	Where should teams go?
POST	/api/enforcement/{id}/action	{"status": "deployed"}	Updated record	Mark action deployed
GET	/api/enforcement/stats	—	Counts by status	Dashboard metrics

6.3 Advisory Endpoints

Method	Endpoint	Body	Response	Description
POST	/api/advisory/ask	{"question": "...", "language": "bn", "lat": 22.57, "lon": 88.36}	{"answer": "...", "aqi": 245, "category": "Poor"}	Health advisory chatbot
GET	/api/advisory/templates	?language=bn	Pre-written advisories by AQI range	Quick lookup

7. AI/ML Architecture

7.1 Forecasting Engine: XGBoost Regressor

Why XGBoost over NeuralProphet/LSTM:

- Trains in seconds, not hours
- Handles tabular time-series features natively
- Built-in feature importance (explainable to judges)
- Easier to validate and debug solo
- No GPU needed

Target Variables:

- AQI(t+24h) — primary
- AQI(t+48h) — secondary
- AQI(t+72h) — tertiary

Feature Engineering:

Feature Category	Features	Rationale
Lagged AQI	AQI(t-1h), AQI(t-6h), AQI(t-12h), AQI(t-24h), AQI(t-48h)	Autoregressive component
Rolling Stats	mean_6h, mean_24h, std_24h	Trend and volatility
Weather	temp_c, humidity_pct, wind_speed, wind_dir_sin, wind_dir_cos, pressure	Dispersion and trapping
Temporal	hour, day_of_week, month, is_weekend, is_rush_hour	Traffic and activity patterns
Seasonal	is_winter, is_festival_window, is_stubble_season	Kolkata-specific: Oct-Jan inversion, Diwali, NW wind biomass
Spatial	station_lat, station_lon (as proxy for local geography)	Howrah vs. Salt Lake vs. Barrackpore

Training Pipeline:

1. Pull historical CPCB data (minimum 30 days, ideally 90+)
2. Join with weather data by timestamp
3. Create feature matrix with lags
4. Split: 80% train, 20% test (time-based split, not random)
5. Train XGBoost with early stopping
6. Validate against **persistence baseline**: `prediction = last_known_AQI`
7. Report: `RMSE_model`, `RMSE_baseline`, `MAE_model`, R^2

Inference Pipeline:

1. Pull latest 48h of readings + weather
2. Compute feature vector
3. Predict 24h, 48h, 72h horizons
4. Compute confidence intervals via quantile regression or bootstrap
5. Store in `forecasts` table
6. Serve via API

Expected Performance: Beating persistence baseline by 15–25% on RMSE is a legitimate, defensible result. Do not inflate.

7.2 Source Attribution Engine

Type: Explainable heuristic scoring model (not black-box ML).

Inputs per Ward:

- `current_aqi`: Real-time interpolated AQI
- `road_density`: From OSM (km road / km²)
- `industrial_score`: Proximity to OSM industrial zones (0-100)
- `construction_count`: OSM building=construction tags within ward
- `satellite_no2`: Sentinel-5P NO₂ anomaly for ward centroid
- `wind_dir`: Current wind direction (degrees)
- `wind_speed`: Current wind speed
- `time_of_day`: Hour (0-23)
- `month`: Month (1-12)

Scoring Algorithm:

```
def attribute_sources(ward, aqi, weather, time_features, satellite_no2):
    scores = {}

    # Traffic: peaks during rush hour, correlates with road density
    scores['traffic'] = (
        ward.road_density * 0.25 +
        (1.0 if time_features.is_rush_hour else 0.3) * 0.20 +
        (0.8 if aqi > 200 else 0.4) * 0.15
    )

    # Industrial: satellite NO2 + proximity to industrial zones
    scores['industrial'] = (
        ward.industrial_score * 0.30 +
        satellite_no2 * 0.25 +
        (0.5 if wind_speed < 5 else 0.2) * 0.10 # trapping condition
    )

    # Construction: active sites + PM10 dominance
    scores['construction'] = (
        min(ward.construction_count / 10, 1.0) * 0.35 +
        (0.3 if pm10 > pm25 * 1.5 else 0.1) * 0.15
    )

    # Biomass: winter + NW wind (Gangetic drift from Punjab/Haryana)
    scores['biomass'] = (
        (0.6 if time_features.is_winter else 0.1) * 0.30 +
        (0.4 if 290 <= weather.wind_dir <= 340 else 0.1) * 0.25 +
        (0.3 if aqi > 300 and time_features.is_morning else 0.1) * 0.15
    )

    # Residential: baseline, higher in evening cooking hours
    scores['residential'] = (
        0.15 + # baseline
        (0.2 if 17 <= time_features.hour <= 21 else 0.05) * 0.10
    )
```

```

# Normalize to sum = 100%
total = sum(scores.values())
for key in scores:
    scores[key] = (scores[key] / total) * 100

primary = max(scores, key=scores.get)
confidence = 0.5 + (scores[primary] / 100) * 0.5 # 0.5 to 1.0

return {
    'breakdown': scores,
    'primary_source': primary,
    'confidence': confidence,
    'explanation': generate_explanation(primary, scores, ward, weather)
}

```

Why this is legitimate: This is how real source apportionment is approximated before full chemical transport modelling. The model is fully explainable — every score has a documented rationale. Judges can probe any number and you can defend it.

7.3 Enforcement Prioritization Algorithm

```

def compute_priority(ward, forecast, attribution):
    # Severity: how bad is it now + how bad will it be?
    severity = (ward.current_aqi * 0.3) + (forecast.predicted_aqi_24h * 0.4)

    # Exposure: how many people affected?
    exposure = (
        (ward.population / 100000) * 0.15 +
        (ward.school_count * 2) * 0.05 +
        (ward.hospital_count * 3) * 0.05
    )

    # Actionability: can we actually do something?
    actionability = {
        'construction': 0.9, # Can halt work immediately
        'industrial': 0.7,   # Can inspect, reduce load
        'traffic': 0.6,      # Can divert, enforce odd-even
        'biomass': 0.3,      # Hard to control quickly
        'residential': 0.2   # Very hard to control
    }.get(attribution.primary_source, 0.5)

    # Trend: is it getting worse?
    trend = 1.0 if forecast.predicted_aqi_24h > ward.current_aqi else 0.7

    # Composite score (0-100)
    priority = (
        severity * 0.35 +
        exposure * 0.25 +
        actionability * 20 * 0.20 + # scaled to 0-18
        trend * 10 * 0.20          # scaled to 0-12
    )

```

```
return min(priority, 100)
```

7.4 RAG Advisory Bot

Architecture:

1. **Corpus:** Parse CPCB health advisories, WHO guidelines, Kolkata-specific action plans into chunks
2. **Embed:** OpenAI text-embedding-3-small → store in documents table (or pgvector in production)
3. **Retrieve:** User query → embed → similarity search top-3 chunks
4. **Generate:** GPT-4o-mini with structured system prompt

System Prompt:

```
You are AETHER, an air quality health advisor for Kolkata residents.
Answer based ONLY on the provided context documents.
If the user asks in Bengali, answer in Bengali.
If the user asks in Hindi, answer in Hindi.
If the user asks in English, answer in English.
Include specific AQI numbers and actionable advice.
Be concise (2-3 sentences max).
Current location AQI: {aqi}
Forecast: {forecast_summary}
Vulnerable nearby: {schools} schools, {hospitals} hospitals
```

Context Injection:

- Current AQI for user's lat/lon
- 24h forecast summary
- Ward vulnerability (schools, hospitals nearby)
- Time of day (morning exercise vs. evening)

8. Data Sources (Exact URLs & Methods)

8.1 CPCB Live AQI Data (Primary)

Registration: <https://data.gov.in> → Sign up → Request CPCB AQI API key

API Endpoint: <https://api.data.gov.in/resource/3b01bcb8-0b14-4abf-b6f2-c1bfd384ba69>

Parameters: api-key={KEY}&format=json&filters[city]=Kolkata&limit=1000

Returns: Station name, city, lat/lon, pollutant values (PM2.5, PM10, NO , SO , CO, O), timestamps

Update Frequency: Hourly

Kolkata Stations: Rabindra Bharati, Victoria Memorial, Fort William, Jadavpur University, Bidhan-nagar, Howrah, Durgapur, Barrackpore, Asansol

Fallback: If API key is delayed, use CPCB CCR portal https://app.cpcbcr.com/aqi_dashboard/ for manual CSV export, or GitHub community wrappers.

8.2 Weather Data (No API Key Required)

Source: Open-Meteo

Endpoint: <https://api.open-meteo.com/v1/forecast>

Parameters: latitude=22.5726&longitude=88.3639&hourly=temperature_2m,relative_humidity_2m,wind_speed_10m,win

Returns: Hourly weather data for past 7 days + next 7 days

Cost: Free, unlimited, no registration

8.3 Ward Boundaries

Source: Datameet / OpenCity Initiative

Search: "kolkata municipal corporation ward boundary geojson"

Alternative: https://github.com/datameet/indian_open_data

Format: GeoJSON with 144 ward polygons

Processing: Parse with geopandas, compute centroids, store as JSON string in SQLite

8.4 Satellite NO₂ / Aerosol

Source: Google Earth Engine → Sentinel-5P TROPOMI

Registration: <https://code.earthengine.google.com> (free for research/education)

Datasets: - COPENICUS/S5P/0FFL/L3_N02 (Nitrogen Dioxide) - COPENICUS/S5P/0FFL/L3_AER_AI (UV Aerosol Index)

- COPENICUS/S5P/0FFL/L3_N02 (Nitrogen Dioxide)
- COPENICUS/S5P/0FFL/L3_AER_AI (UV Aerosol Index) **Resolution:** ~5.5km × 3.5km (state this honestly)
- Use:** Corroborating signal for industrial attribution, not primary AQI measurement

8.5 OpenStreetMap (Land Use)

Tool: osmnx Python package or Overpass API

Queries: - Roads: highway~"motorway|trunk|primary|secondary" in bounding box

- Roads: highway~"motorway|trunk|primary|secondary" in bounding box
- Industrial: landuse=industrial in bounding box
- Construction: building=construction in bounding box
- Schools: amenity=school
- Hospitals: amenity=hospital **Bounding Box for Kolkata:** 22.45, 88.15, 22.65, 88.55 (SW lat, SW lon, NE lat, NE lon)

9. Frontend Page Structure

9.1 /dashboard — Live Situation Room (Primary View)

Layout: Full-screen map (75%) + collapsible side panel (25%)

Map Layers (toggle-able):

- Base: CartoDB Dark Matter tiles

- Layer 1: Ward boundaries (subtle white lines, 144 wards)
- Layer 2: Station markers (colored circles: green→maroon)
- Layer 3: Ward choropleth (interpolated AQI coloring)
- Layer 4: Heatmap (smooth gradient between stations)
- Layer 5: Satellite NO₂ overlay (toggle, semi-transparent)

Side Panel (on ward click):

- Ward name + current AQI badge (color-coded)
- Primary source attribution with confidence
- 24h forecast sparkline (mini chart)
- “View Full Forecast” button → navigates to /forecast
- Top recommended action (one line)
- “Mark Deployed” button

Top Bar:

- City selector (Kolkata default, Delhi/Mumbai in dropdown)
- Last updated timestamp
- City average AQI badge
- Refresh button

Bottom Ticker:

- Scrolling: “Latest: Enforcement team deployed to Ward 57 (Howrah) — Construction halt recommended”

9.2 /forecast — Deep Dive

Layout: Location search + forecast table + trend chart

Components:

- Search bar: “Enter area or lat,lon”
- Location card: Current AQI, category, primary source
- Forecast chart: Recharts area chart, 72 hours, confidence band shaded
- Forecast table: Hour-by-hour breakdown (AQI, category, wind, temp)
- Comparison toggle: “What if no action?” vs “With intervention” (simulated scenario)
- Export: Download CSV / PNG

9.3 /compare — Multi-City Intelligence

Layout: 3 city cards + ranking table

City Cards:

- Kolkata (primary, full data)

- Delhi (secondary)
- Mumbai (secondary)

Each card shows:

- Current city-wide average AQI
- 24h trend sparkline
- Top source category
- Number of active enforcement actions
- “View Dashboard” link

Ranking Table:

- Cities by current AQI (worst to best)
- Improvement rate (if historical data available)
- Intervention effectiveness score

9.4 /advisory — Citizen Interface

Layout: Chat interface (WhatsApp-style)

Components:

- Language selector: English / (Bengali) / (Hindi)
- Location auto-detect (browser geolocation) or manual input
- Chat bubbles: User (right), AETHER (left, with AQI badge)
- Quick questions: “Is it safe to jog?”, “Should kids go to school?”, “Which areas to avoid?”
- Response cards: Structured with AQI number, health risk level, specific actions, duration

10. Day-by-Day Build Plan

Phase 1: Foundation (Days 1–2)

Day 1 — Scaffold & Registration

- Register `data.gov.in` account and request CPCB API key
- Register Google Earth Engine account
- Create GitHub repository `aether-aqi`
- Scaffold Next.js frontend: `npx create-next-app@latest aether-frontend --typescript --tailwind --eslint --app`
- Install frontend dependencies: `npm install react-leaflet leaflet recharts axios lucide-react`
- Initialize shadcn/ui: `npx shadcn-ui@latest init` and add components: `card`, `button`, `badge`, `select`, `scroll-area`, `sheet`, `table`

- Scaffold FastAPI backend: `mkdir aether-backend && cd aether-backend && python -m venv venv && source venv/bin/activate`
- Install backend dependencies: `pip install fastapi uvicorn sqlalchemy alembic python-dotenv requests pandas numpy scikit-learn xgboost shap geopandas osmnx`
- Create `.env.example` files for both frontend and backend
- **Deliverable:** Both repos run locally. Frontend on `localhost:3000`. Backend on `localhost:8000` with `/docs` showing.

Day 2 — Database & Data Pipeline

- Create `backend/models.py` with all SQLAlchemy models (stations, readings, weather, wards, forecasts, attributions, enforcement_queue)
- Create SQLite database with `create_all()`
- Build `backend/scripts/seed_stations.py` — manually seed Kolkata's 10 CPCB stations with known lat/lon
- Build `backend/services/fetch_cpcb.py` — fetch from CPCB API or fallback CSV, parse, insert into readings
- Build `backend/services/fetch_weather.py` — fetch from Open-Meteo, insert into weather
- Run both fetchers, verify database has real data
- Build `backend/api/health.py` and `backend/api/stations.py` endpoints
- **Deliverable:** SQLite file contains 24+ hours of real AQI and weather data. API returns live station list.

Phase 2: Live Map (Days 3–4)

Day 3 — Map & Markers

- Build `frontend/components/AetherMap.tsx` — React-Leaflet with CartoDB Dark Matter tiles
- Build `frontend/components/StationMarker.tsx` — color-coded circles (CPCB color scale)
- Build `frontend/app/dashboard/page.tsx` — full-screen map layout
- Build `backend/api/aqi.py` — GET `/api/aqi/live` returns latest readings with station info
- Connect frontend to backend, verify live markers appear on Kolkata map
- Add popup on marker click: station name, AQI, last updated, pollutant breakdown
- **Deliverable:** `localhost:3000/dashboard` shows dark map of Kolkata with real, live-colored station dots.

Day 4 — Ward Boundaries & Interpolation

- Download KMC ward GeoJSON (144 wards)
- Build `backend/scripts/seed_wards.py` — parse GeoJSON, compute centroids, insert into `wards` table
- Build interpolation function: for each ward center, find 3 nearest stations, compute inverse-distance-weighted AQI

- Build `backend/api/aqi.py` — GET `/api/aqi/heatmap` returns ward grid with interpolated AQI
- Add GeoJSON layer to map: subtle white ward boundaries
- Add choropleth layer: color wards by interpolated AQI
- Build click interaction: click ward → fly-to animation + side panel opens
- **Deliverable:** Click anywhere in Kolkata → see interpolated AQI for that ward. Side panel shows ward summary.

Phase 3: Forecasting (Days 5–7)

Day 5 — Model Training (Part 1)

- Export historical data from DB to CSV (or use manual CPCB historical export)
- Create `backend/notebooks/01_feature_engineering.ipynb`
- Engineer all features: lagged AQI, rolling stats, weather, temporal, seasonal, spatial
- Create train/test split (time-based, last 7 days held out)
- Train first XGBoost model for AQI(t+24h)
- Compute persistence baseline: RMSE if prediction = last known AQI
- **Deliverable:** Notebook shows model trained, feature importance plot, RMSE vs baseline.

Day 6 — Model Training (Part 2) + API Integration

- Train models for 48h and 72h horizons (or use same model with different target)
- Add quantile regression for confidence intervals (or bootstrap method)
- Build `backend/services/forecaster.py` — model wrapper class
- Build `backend/api/forecast.py` — GET `/api/forecast?lat=&lon=&hours=72`
- Build `frontend/components/ForecastChart.tsx` — Recharts area chart with confidence band
- Integrate into side panel: click ward → show 72h forecast chart
- **Deliverable:** Click a ward → side panel shows 72h forecast with confidence intervals.

Day 7 — Model Validation & Refinement

- Compute full validation metrics: RMSE, MAE, R², MAPE for 24h/48h/72h
- Generate validation report (save as markdown for deck)
- Test edge cases: missing data, new station, extreme AQI values
- Add model retraining script: `backend/scripts/retrain.py` (run nightly via cron)
- **Deliverable:** Documented accuracy numbers ready for presentation. Model handles edge cases gracefully.

Phase 4: Attribution & Intelligence (Days 8–9)

Day 8 — Source Attribution

- Build `backend/services/attributor.py` — implement scoring algorithm from Section 7.2

- Fetch OSM data for Kolkata: roads, industrial zones, construction sites, schools, hospitals
- Populate ward features: `road_density`, `industrial_score`, `construction_count`, etc.
- Build `backend/api/attribution.py` — GET `/api/attribution/{ward_id}`
- Build `frontend/components/SourceBreakdown.tsx` — horizontal bar chart
- Build explanation generator: human-readable text from scores
- Integrate into side panel: click ward → show source breakdown
- **Deliverable:** Click a ward → “Likely source: Traffic (58%), Industrial (22%), Construction (12%)...”

Day 9 — Enforcement Queue

- Build `backend/services/enforcement_scorer.py` — implement priority algorithm from Section 7.3
- Build `backend/api/enforcement.py` — GET `/api/enforcement`, POST `/api/enforcement/{id}/action`
- Build `frontend/app/enforcement/page.tsx` — sortable table view
- Build `frontend/components/EnforcementPanel.tsx` — ranked list with justification, “Deploy” button
- Add status changes: open → deployed → resolved
- Add stats endpoint: counts by status for dashboard metrics
- **Deliverable:** Dashboard shows ranked action list. Click “Deploy” → status updates.

Phase 5: Advisory & Multi-City (Days 10–11)

Day 10 — Advisory Bot

- Build `backend/services/rag.py` — LangChain pipeline with OpenAI embeddings
- Ingest corpus: CPCB health advisories, WHO guidelines (parse 3-5 key documents)
- Store embeddings in `documents` table
- Build `backend/api/advisory.py` — POST `/api/advisory/ask`
- Build `frontend/app/advisory/page.tsx` — chat interface
- Add Bengali language support: pre-written templates + LLM translation
- Add quick question buttons: “Safe to jog?”, “School today?”, “Areas to avoid?”
- **Deliverable:** Chatbot answers questions in English and Bengali with specific AQI context.

Day 11 — Multi-City & Satellite

- Seed Delhi stations (10+ CPCB stations) and Mumbai stations (10+ CPCB stations)
- Run data fetchers for Delhi and Mumbai (same pipeline, different city param)
- Build `frontend/app/compare/page.tsx` — city cards + ranking table
- Pull Sentinel-5P NO₂ tile for Kolkata via Google Earth Engine
- Add satellite overlay as toggle-able layer on map
- Add loading states, empty states, error handling across all pages

- **Deliverable:** 3 cities selectable. Satellite overlay works. UI is polished.

Phase 6: Deploy & Present (Days 12–13)

Day 12 — Deployment

- Deploy frontend to Vercel: `vercel --prod`
- Deploy backend to Render or Railway (or use ngrok as fallback)
- Set environment variables: `DATABASE_URL`, `OPENAI_API_KEY`, `CPCB_API_KEY`
- Test live URL end-to-end
- Create local Docker fallback: `docker-compose.yml` with frontend + backend + SQLite
- **Deliverable:** Live public URL. Local fallback ready.

Day 13 — Presentation & Demo Video

- Build 8-slide deck:
 1. Problem (cite 1.67M deaths, 31% cities with no action protocol)
 2. Solution (AETHER screenshot)
 3. Architecture (3-layer diagram)
 4. Live Demo (screenshots as backup)
 5. Model Validation (real RMSE vs baseline numbers)
 6. Kolkata Deep-Dive (local context: Howrah, brick kilns, winter inversion)
 7. Multi-City + Scale
 8. Roadmap (what's real today, what's next)
- Record 5-minute demo video with voiceover (script in Section 12)
- Upload video to YouTube/Google Drive
- **Deliverable:** PDF deck + MP4 demo video.

11. File/Folder Structure

```
aether/
├─ README.md
├─ .env.example
├─ .gitignore
├─
├─ backend/
│   └─ app/
│       ├── __init__.py
│       ├── main.py           # FastAPI entry, CORS, router inclusion
│       ├── config.py        # Pydantic settings, env vars
│       ├── database.py      # SQLAlchemy engine, session, base
│       ├── models.py        # All SQLAlchemy ORM models
│       └── schemas.py       # Pydantic request/response models
```

```

| | | | |
| | | | |└─ api/
| | | | |    |
| | | | |    |└─ health.py
| | | | |    |└─ stations.py
| | | | |    |└─ aqi.py          # Live readings, heatmap
| | | | |    |└─ forecast.py     # ML predictions
| | | | |    |└─ attribution.py  # Source scoring
| | | | |    |└─ enforcement.py  # Priority queue
| | | | |    |└─ advisory.py     # RAG chatbot
| | | | |    |└─ cities.py       # City config
| | | | |    |
| | | | |    |└─ services/
| | | | |    |    |
| | | | |    |    |└─ fetch_cpcb.py
| | | | |    |    |└─ fetch_weather.py
| | | | |    |    |└─ fetch_osm.py
| | | | |    |    |└─ forecaster.py  # XGBoost wrapper
| | | | |    |    |└─ attributor.py  # Source scoring logic
| | | | |    |    |└─ enforcement_scorer.py
| | | | |    |    |└─ rag.py         # LangChain pipeline
| | | | |    |
| | | | |    |└─ scripts/
| | | | |    |    |
| | | | |    |    |└─ seed_stations.py
| | | | |    |    |└─ seed_wards.py
| | | | |    |    |└─ retrain.py     # Nightly model retrain
| | | | |    |    |└─ fetch_all.py   # Manual data refresh
| | | | |    |
| | | | |    |└─ notebooks/
| | | | |    |    |
| | | | |    |    |└─ 01_eda.ipynb
| | | | |    |    |└─ 02_feature_engineering.ipynb
| | | | |    |    |└─ 03_model_validation.ipynb
| | | | |    |
| | | | |    |└─ requirements.txt
| | | | |    |└─ Dockerfile
| | | | |    |└─ alembic.ini
| | | | |
| | | | |└─ frontend/
| | | | |    |
| | | | |    |└─ app/
| | | | |    |    |
| | | | |    |    |└─ layout.tsx          # Root layout, providers, dark theme
| | | | |    |    |└─ page.tsx           # Landing → redirect to /dashboard
| | | | |    |    |└─ globals.css
| | | | |    |    |
| | | | |    |    |└─ dashboard/
| | | | |    |    |    |
| | | | |    |    |    |└─ page.tsx       # Main map view
| | | | |    |    |└─ forecast/
| | | | |    |    |    |
| | | | |    |    |    |└─ page.tsx       # Deep dive
| | | | |    |    |└─ compare/
| | | | |    |    |    |
| | | | |    |    |    |└─ page.tsx       # Multi-city
| | | | |    |    |└─ advisory/
| | | | |    |    |    |
| | | | |    |    |    |└─ page.tsx       # Chatbot
| | | | |    |
| | | | |    |└─ components/

```

```

| | |─ ui/                # shadcn components (auto-generated)
| | |─ map/
| | |   │─ AetherMap.tsx
| | |   │─ HeatmapLayer.tsx
| | |   │─ WardBoundary.tsx
| | |   └─ StationMarker.tsx
| | |─ charts/
| | |   │─ ForecastChart.tsx
| | |   │─ SourceBreakdown.tsx
| | |   └─ TrendSparkline.tsx
| | |─ dashboard/
| | |   │─ SidePanel.tsx
| | |   │─ CityHeader.tsx
| | |   └─ EnforcementTicker.tsx
| | |─ advisory/
| | |   │─ ChatInterface.tsx
| | |   │─ LanguageSelector.tsx
| | |   └─ QuickQuestions.tsx
| | └─ compare/
| |     └─ CityCard.tsx
| |
| └─ lib/
| |   │─ api.ts           # Axios client, typed requests
| |   │─ utils.ts         # cn() helper
| |   └─ constants.ts     # City configs, AQI breakpoints, colors
| |
| └─ types/
| |   └─ index.ts         # TypeScript interfaces
| |
| └─ public/
| |   └─ geojson/
| |       │─ kolkata-wards.json
| |       │─ delhi-wards.json
| |       └─ mumbai-wards.json
| |
| └─ tailwind.config.ts
| └─ next.config.js
| └─ package.json
|
└─ docs/
    │─ architecture-diagram.png
    │─ demo-script.md
    │─ presentation.pdf
    └─ demo-video.mp4

```

12. Demo Strategy

12.1 Judging Criteria Alignment

Criteria (Weight)	How AETHER Delivers
Innovation (25%)	Real multi-source fusion (ground stations + satellite + weather + OSM) into explainable attribution + forecast. Not just another AQI dashboard.
Business Impact (25%)	Directly addresses CPCB finding: only 31% of cities have actionable response protocols. Enforcement ranking turns data into deployment orders.
Technical Excellence (20%)	Real trained XGBoost with validated RMSE vs baseline. Real geospatial interpolation. Real satellite integration. All independently verifiable.
Scalability (15%)	Multi-city module proves pipeline isn't hardcoded. Clean separation of ingestion/storage/intelligence/frontend layers.
User Experience (15%)	One coherent map-first interface. Mobile-responsive. Multi-language (English + Bengali). No training required.

12.2 Demo Video Script (5 Minutes)

[0:00–0:30] Hook — The Problem

“I live in Kolkata. In winter, my city hits AQI 400 — not just from stubble burning, but from a specific cocktail of Howrah Bridge corridor traffic, brick kiln emissions in the northeast, and the Gangetic inversion layer trapping it all. AETHER knows the difference.”

[0:30–1:30] Live Dashboard

- Show dark map of Kolkata with real CPCB station dots
- Point to a red zone (e.g., Ward near Howrah)
- Click → side panel opens
- “This ward is at 287 right now. Here’s why: 62% traffic, 22% industrial.”
- Point to wind direction indicator: “Wind is coming from the industrial corridor.”

[1:30–2:30] Forecasting Intelligence

- Show 72-hour forecast chart
- “NeuralProphet isn’t needed — XGBoost on real features predicts tomorrow at 340.”
- Show confidence band: “We predict with 85% confidence it’ll stay in the Red zone.”
- Mention validation number: “Our model beats ‘tomorrow equals today’ by 22% on RMSE.”

[2:30–3:30] Enforcement Action

- Switch to enforcement ranking view
- “If I’m the WBPCB officer this morning, my three teams go here, here, and here — ranked by severity, population exposure, and whether we can actually do something.”
- Click top item: “Ward 57, Howrah. Construction halt. 94 priority score.”

- [3:30–4:15] Citizen Advisory**

- ### [4:15–4:45] Multi-City & Scale

- [4:45–5:00] Closing**

- ### 12.3 Q&A Preparation (What Judges Will Ask)

A: “Real CPCB station data via data.gov.in API, refreshed hourly. Real Open-Meteo weather. Real Sentinel-5P satellite via Google Earth Engine. We held out 7 days of real data to validate the forecast model.”

A: “Our XGBoost model achieves RMSE of [X] versus a persistence baseline of [Y] — a [Z]% improvement. We validate against held-out real data, not training data.”

A: “We use an explainable heuristic model combining satellite NO_x, OpenStreetMap land use, wind direction, and time-of-day patterns. It’s how real source apportionment is approximated before full chemical transport modelling. Every score is defensible.”

A: “We already run it for Delhi and Mumbai. Adding a new city requires only: CPCB station list, ward boundaries, and the same data fetchers. The model architecture is city-agnostic.”

A: “We interpolate between real stations using inverse-distance weighting, adjusted by satellite NO₂. True 1km atmospheric dispersion modelling requires computational fluid dynamics — that’s a research project, not a hackathon scope. Our approach is honest and defensible.”

Generated by Kimi.ai

Risk	Probability	Mitigation
data.gov.in API key delayed	Medium	Register Day 1. Fallback: manual CSV export from CPCB CCR portal. Community GitHub wrappers available.
Sparse historical data for model	Medium	Pull as much as possible. If <30 days, use simpler model (linear regression with seasonality) and be honest about it.
Ward GeoJSON unavailable for Kolkata	Low	Search Datameet, OpenCity, KMC portal. Fallback: use 1km grid cells instead of wards.
Google Earth Engine approval slow	Medium	Register Day 1. Fallback: skip satellite layer for v1, add as “coming soon” in deck.
Model overfits / suspicious accuracy	Low	Always report against held-out test set and persistence baseline. Honest modesty > inflated claims.
Map slow with many layers	Low	Make layers toggle-able. Default to 2 visible. Use server-side clustering if needed.
Deployment issues	Medium	Always have local Docker compose + ngrok fallback. Record demo video as insurance.
OpenAI API costs / rate limits	Low	Use GPT-4o-mini (\$0.15/1M input). For hackathon, \$5 credit is plenty. Fallback: rule-based templates.
Scope creep	High	Revisit Section 2 daily. If behind, cut satellite layer or multi-city depth before cutting core features.

14. Deliverables Checklist

14.1 Working Prototype

- Live hosted URL (Vercel + Render/Railway)
- Local Docker fallback ready
- Must demonstrate: live map, ward click → attribution, forecast chart, enforcement ranking, advisory chatbot, multi-city toggle

14.2 Architecture Diagram

- 3-layer flow: Ingestion → Storage → Intelligence → Frontend
- LLM narration layer clearly marked as AUXILIARY, not primary
- Data sources labeled with real names (CPCB, Open-Meteo, Sentinel-5P, OSM)

14.3 Presentation Deck (8 slides)

- Slide 1: Problem + human cost stats
- Slide 2: Solution (one screenshot)
- Slide 3: Architecture diagram
- Slide 4: Live demo screenshots (backup)
- Slide 5: Model validation (RMSE vs baseline — real numbers)
- Slide 6: Kolkata deep-dive (local context)
- Slide 7: Multi-city + scale proof
- Slide 8: Roadmap (what's real, what's next)

14.4 Demo Video

- 5-minute screen capture with voiceover
- Follow script from Section 12.2 exactly
- Upload to YouTube or Google Drive
- Test playback on mobile before submission

15. Quick Reference: AQI Color Breakpoints (CPCB India)

AQI Range	Category	Color Code	Hex
0–50	Good	Green	#009966
51–100	Satisfactory	Light Green	#92D050
101–200	Moderate	Yellow	#FFCC00
201–300	Poor	Orange	#FF9933
301–400	Very Poor	Red	#CC0033
401–500	Severe	Maroon	#660099

Use these exact colors for all map markers, ward choropleths, and UI badges. Consistency builds credibility.

16. Environment Variables (.env.example)

```
# Backend
DATABASE_URL=sqlite:///./aether.db
CPCB_API_KEY=your_data_gov_in_key_here
OPENAI_API_KEY=your_openai_key_here
OPENWEATHER_API_KEY=optional_fallback
GEE_PROJECT=your_earth_engine_project

# Frontend
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_MAP_TILE_URL=https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png
```


17. Final Notes for Collaborating AI Assistants

Context for AI tools reading this document:

This is a concrete, executable build plan for a solo developer vibe-coding a hackathon project. Every section contains specific, actionable details — not vague suggestions. When generating code:

1. **Prioritize working code over perfect architecture.** The builder is solo. A single working file is better than a perfectly modular system that doesn't run.
2. **Use the exact tech stack listed.** Do not suggest alternatives unless the specified tool is genuinely broken.
3. **Kolkata-specific context is a feature, not a constraint.** Use local knowledge (Howrah, New Town, winter inversion, brick kilns) to make the demo authentic.
4. **The LLM is a narrator, not a calculator.** All AQI numbers, forecasts, and scores must come from deterministic Python code. The LLM only writes explanations.
5. **Validate against real data.** Never suggest synthetic data as a primary source. The CPCB API is the ground truth.
6. **Keep it deployable.** Every component should work locally AND be ready for Vercel/Render deployment.

If you are an AI assistant continuing this build:

- Start with the Day 1 scaffold (Section 10, Phase 1)
- Generate complete, copy-pasteable files
- Include error handling and comments
- Test mentally before presenting code
- Ask for clarification only on ambiguous requirements, not on items already specified in this document

End of Document